HONEYWELL SOFTWARE ENGINEERING TEAMS — IA · BA · PA&T

AI Champions Programme — GitHub Copilot Enterprise, SDD, MCP & Agentic Software Engineering

Module 03

Prompt Engineering Recap & Context Engineering

A quick recap of prompt engineering fundamentals — then a deep dive into context engineering: choosing, layering, compressing and scoping what the model sees, so Copilot gets it right with fewer iterations.

DURATION
2 hours

FORMAT
Recap + progressive lab

PRIMARY TOOL
Copilot Enterprise in VS Code

How We'll Spend These Two Hours

Prompting basics first, then most of the time on context engineering — ending in a measured lab.



01

Prompt Fundamentals & Techniques

20 min

Anatomy of a good prompt, the technique ladder, decompose–iterate–verify.



02

From Prompting to Context Engineering

15 min

Where prompts hit a ceiling, what context is, and the seven sources Copilot can draw on.



03

The Four Moves: Select · Layer · Compress · Scope

30 min

The core toolkit for deciding what goes into the context window — and what stays out.



04

Failure Modes & Token Discipline

15 min

Poisoning, distraction, confusion, clash, lost-in-the-middle — and how to budget tokens.



05

Specs, Agents & Best-Practice Checklist

15 min

Specs and tests as context, agentic retrieval, and a checklist you can pin to the wall.



06

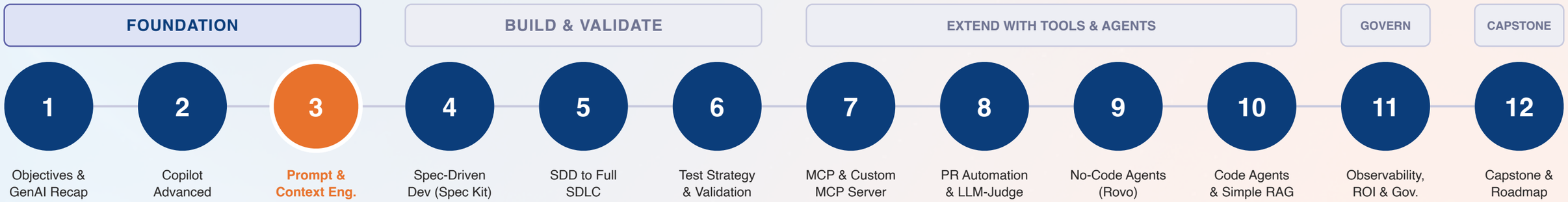
Lab: Prompt-Only to Engineered Context

25 min

Five rounds on one task. Measure tokens, iterations, time and quality at each step.

Module 03 in the 12-Module Journey

The skill that decides whether every later module — specs, agents, MCP — works well or poorly.



M02

Instructions, prompt files and # mentions are the tools you use today

M04–05

Specs are the most structured form of context you will write

M07

MCP is context delivered just-in-time from external systems

M11

Token and cost discipline from today feeds observability and ROI

Why this matters: the model is only as good as the context it is given. Context engineering is the discipline of giving it exactly the right information — no more, no less.

Basics: Build a Prompt Layer by Layer

Same task, five prompts. Each layer removes one source of ambiguity.

1	**Task**	`Write a function to validate emails.`
2	**+ Role & goal**	`As a senior C# engineer, write an email validator for our signup API.`
3	**+ Constraints**	`…use .NET 8, no regex, return a Result<T>, no external packages.`
4	**+ Example**	`…e.g. "a@b.co" → Ok; "a@@b" → Error("double @").`
5	**+ Output format**	`…return only the method and 3 xUnit tests, no prose.`

WHAT EACH LAYER BUYS YOU

Task	Says what to produce
Role & goal	Sets level and intent
Constraints	Shrinks the solution space
Example	Shows, not tells, the pattern
Format	Output you can use as-is

Illustrative example: the C#/.NET prompts above are made up for teaching — use your own team's language and domain in the lab.

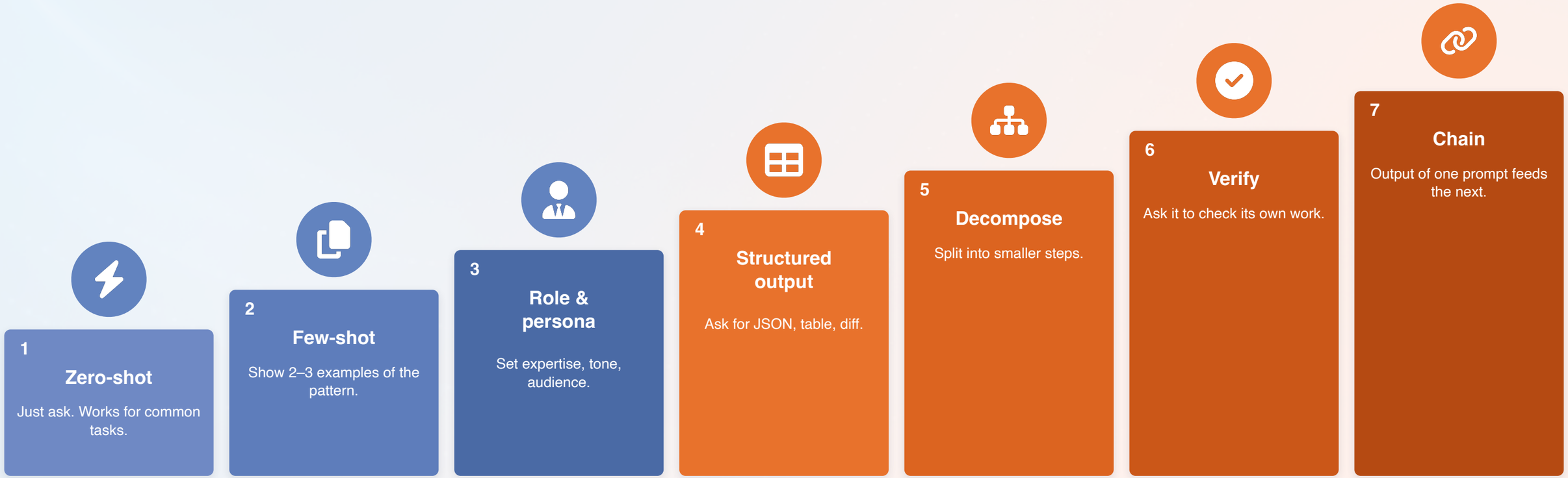
The Prompting Technique Ladder

Seven techniques, from the simplest to patterns that chain several prompts together.

BASIC — shape the request

Cheap, quick wins. Most everyday Copilot chat lives here.

ADVANCED — shape the process



GitHub's Prompt Practices, at a Glance

Eight habits GitHub recommends for Copilot Chat — notice how many are really about context.

1
Start general, then specific
Set the goal, then add detail

2
Give examples
Input/output pairs beat adjectives

3
Break complex tasks down
Smaller prompts, better results

4
Avoid ambiguity
Name the thing you mean

5
Indicate relevant code
Select it, or use # to attach it

6
Experiment and iterate
Refine; don't restart blindly

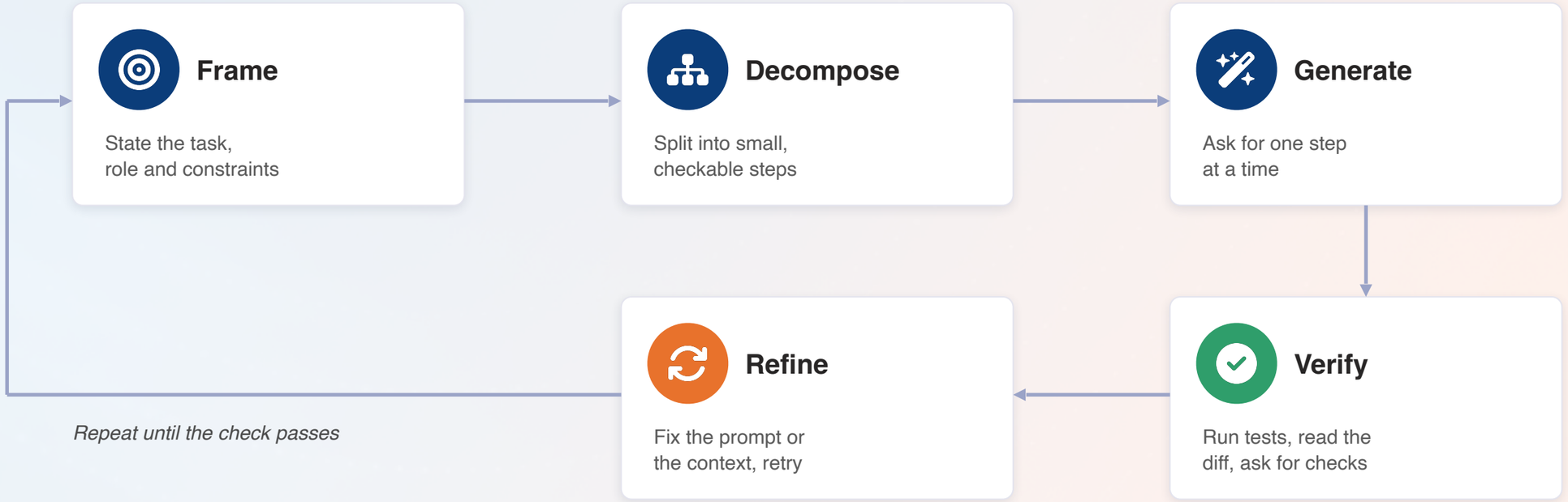
7
Keep history relevant
New thread per task

8
Follow good coding practices
Review everything it produces

Spot the pattern: the two orange cards — “indicate relevant code” and “keep history relevant” — are not prompt wording at all. They are context decisions. That is the doorway to the second half of today.

The Working Loop: Decompose → Iterate → Verify

Good prompting is a loop, not a single message. Each pass narrows the gap to what you actually need.



Rule of thumb: if the third retry still fails, stop rewording the prompt — the model is missing information. Fix the context, not the phrasing.

Where Prompting Hits Its Ceiling

A perfect prompt can still fail. When it does, the cause is usually the information around it.

Missing context

Model guesses.

WHAT YOU SEE

Invents a method name, a config key, or a library your repo doesn't use.

THE FIX

Add the right files, docs or spec.

Wrong context

Model follows stale info.

WHAT YOU SEE

Copies an old pattern, a deprecated API, or an outdated decision from earlier in the chat.

THE FIX

Remove it; refresh from the source of truth.

Too much context

Model loses the thread.

WHAT YOU SEE

Ignores your constraint because it is buried under 40 files and a long history.

THE FIX

Cut to the smallest set that still answers.

The shift: prompt engineering optimises the words of the request. Context engineering optimises the whole set of tokens the model sees when it answers. Same model, very different results.

Prompt Engineering vs Context Engineering

Your prompt is one small slice of what the model actually reads.

WHAT FILLS A COPILOT CHAT REQUEST (ILLUSTRATIVE PROPORTIONS)



Prompt engineering

How do I ask?

- Wording, role, constraints, examples
- One request at a time
- You write it each time

Context engineering

What does the model see?

- Which files, docs, specs and history are in the window
- Persisted, reusable, shared by the team
- Designed once, improved continuously

Guiding principle: find the smallest possible set of high-signal tokens that makes the right outcome likely — adapted from Anthropic's context-engineering guidance.

Seven Context Sources and How Copilot Takes Them In

Left to right: what the model needs → the Copilot mechanism that delivers it → the context window.

SOURCE	COPILOT MECHANISM
Repository code	`#codebase · #file · selection`
APIs & dependencies	`#fetch (docs URL) · #githubRepo`
Architecture & design notes	`docs/ files attached with #file`
Team standards	`.github/copilot-instructions.md`
Issue & PR history	`Issue/PR text pasted or via GitHub tools`
Specs & acceptance criteria	`spec.md attached with #file`
Tests & failing output	`Test files + error output in chat`

CONTEXT WINDOW

A finite budget. Everything above competes for the same space.

Large files: whole → outline → omitted as the window fills.

The Four Moves of Context Engineering

A working toolkit. Every best practice in the rest of this module is one of these four.

SELECT

What goes in?

Pick only the files, docs and facts that bear on this task.

Industry shorthand = select

LAYER

Where does it live?

Put stable rules in instructions, volatile detail in the prompt.

Industry shorthand ≈ write

COMPRESS

How small can it get?

Summarise history, outline files, keep only decisions.

Industry shorthand = compress

SCOPE

What stays out?

One task per chat; push noisy exploration to a separate context.

Industry shorthand ≈ isolate

Order matters: select first (cheapest win), layer so it persists, compress when it grows, scope when tasks multiply. Next four slides take each move in turn.

Move 1 — SELECT: Right Context, Not More Context

Start from the task and work outward. Add a source only if you can say what question it answers.

CANDIDATES IN THE REPO

- `OrderService.cs`
- `OrderController.cs`
- `README.md`
- `OrderServiceTests.cs`
- `Startup.cs`
- `Billing/*.cs`
- `CHANGELOG.md`
- `docs/orders-spec.md`



4 TESTS

Relevant?
Authoritative?
Current?
Small enough?

```
#file selected

OrderService.cs
OrderController.cs
OrderServiceTests.cs
orders-spec.md
```

Implicit vs explicit

Your selection and active file go in automatically. Anything else needs a # mention.

Pin the source of truth

Attach the spec or interface, not a blog-post description of it.

Select a range, not a file

Highlight the method; Copilot treats the selection as the focus.

#codebase is a search

Let Copilot find files — then check what it found before trusting it.

Best practice: name the evidence a good answer depends on, then attach exactly that. If you can't say why a file is attached, detach it.

Move 2 — LAYER: Stable Context First, Volatile Last

Different information changes at different speeds. Store it where its change rate matches.

STABLE — written once, shared by everyone | **VOLATILE — rewritten each task**

Repo-wide instructions

`copilot-instructions.md`

Changes: rarely

Stack, build/test commands, naming, banned patterns

Scoped instructions

`*.instructions.md (applyTo)`

Changes: occasionally

Rules for tests, API layer, UI, SQL

Specs & plan files

`spec.md · plan.md`

Changes: per feature

Goal, acceptance criteria, design decisions

Attached files

`#file · selection`

Changes: per task

The code and tests being changed right now

Your prompt

`Chat message`

Changes: every message

The one specific ask, with its constraint

Best practice: if you typed the same instruction in chat three times, it belongs one layer up. If an instruction applies to one task, it does not belong in an always-on file — it will cost tokens on every request.

Move 3 — COMPRESS: Keep the Signal, Drop the Rest

As chats, files and logs grow, replace raw bulk with distilled versions.

Long chat history	**/compact or Compact Conversation**	**Short summary of decisions** *e.g. 40k → 3k tokens*
Whole large file	**Outline or select the method**	**Signatures + the one method** *e.g. 8k → 600 tokens*
Full build / test log	**Paste failing lines only**	**The error and 5 lines around** *e.g. 12k → 400 tokens*

handoff-note.md

```
## Goal
Add refund support to orders
## Decided
- Use Result<T>, no exceptions
- Refund ≤ original amount
## Done
- RefundService + 3 tests
## Next
- Controller endpoint
- Update spec.md
```

Best practice: compress at natural breakpoints — after a decision, before switching sub-task. Token figures are illustrative; measure your own in the lab. Always keep: goal, decisions, constraints, what's left.

Move 4 — SCOPE: One Task, One Context

Isolation keeps unrelated detail out of the window and keeps noisy exploration from drowning the signal.

NEW CHAT PER TASK, CONNECTED BY FILES

Task A · fresh chat

Design the refund API

Only what this task needs

`plan.md` →

Task B · fresh chat

Implement RefundService

Only what this task needs

`notes.md` →

Task C · fresh chat

Write controller + docs

Only what this task needs

SEND NOISY EXPLORATION TO A SEPARATE CONTEXT

Main agent

Stays focused on the plan

Ask: "where is refund logic?" →

Subagent

Reads dozens of files in its own window →

Short summary returned

Files, findings, open questions — not the raw reads

Best practice: start a new chat when the task changes, when the model starts repeating old mistakes, or after a plan is written to a file. History is context too — keep it relevant.

When Context Goes Wrong: Five Failure Modes

More context is not better context. Know the symptoms so you can diagnose a bad answer quickly.

Poisoning

A wrong fact or hallucination enters the window and gets reused.

→ **Start fresh; correct at the source.**

Distraction

So much history that the model repeats it instead of reasoning.

→ **Compress; new chat.**

Confusion

Irrelevant files or tools pull the answer off target.

→ **Select fewer, sharper sources.**

Clash

Two sources disagree (old spec vs new code); model blends them.

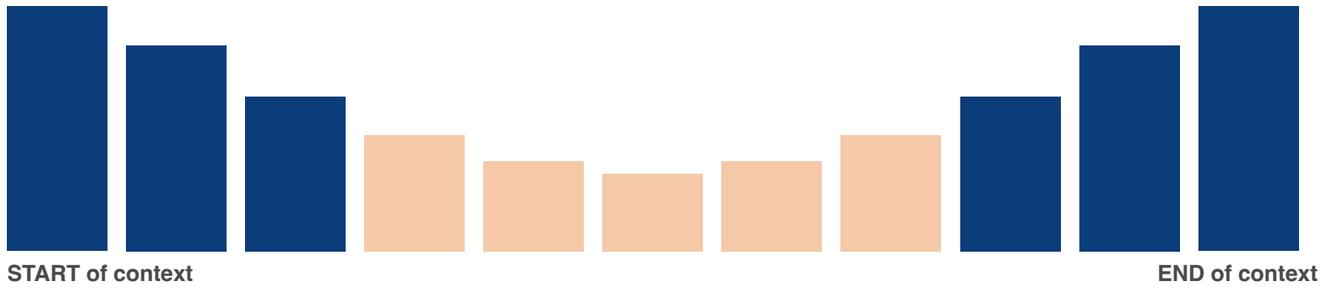
→ **Remove or reconcile one.**

5 · Lost in the middle

Models use information at the start and end of the input best; facts buried in the middle are used less reliably. (Liu et al., TACL, 2024)

→ **Put the critical constraint first or last; repeat it in the prompt.**

Illustrative shape of recall by position — not measured data



Token Discipline: Treat Context as a Budget

Every token you add costs attention and, under usage-based billing, money.

Estimate
What does this task really need?

Budget
Cap files, history, output.

Measure
Watch the context-window indicator.

Adjust
Compress, detach, or start fresh.

READING THE CONTEXT-WINDOW INDICATOR (ILLUSTRATIVE)



Thresholds are a rule of thumb for this course, not a product setting.

WHAT DRIVES TOKEN USE

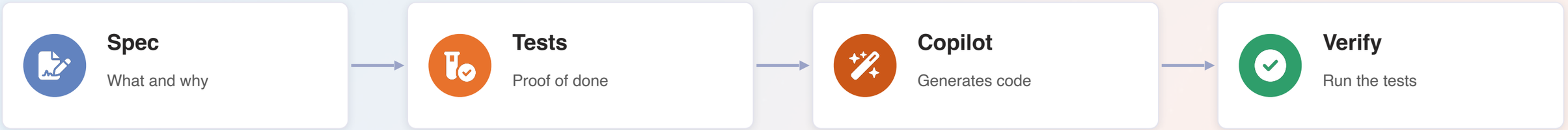
- Always-on instructions
- Attached files
- Chat history
- Tool & search results
- Model output

Why it matters now

- Copilot is billed in AI credits, consumed by token usage (since June 2026).
- Bloated context costs credits on every request, and still lowers answer quality.
- Module 11 turns these habits into measured ROI and governance.

Specs and Tests: The Densest Context You Can Write

A few lines of intent plus a few executable checks tell the model more than pages of prose.



orders-spec.md

```
## Refund
Goal: customers can refund a paid order.

MUST: refund <= original amount
MUST: only for status = Paid
MUST: write an audit entry
OUT OF SCOPE: partial item refunds
```

RefundTests.cs

```
[Fact] void Refund_Over_Amount_Fails()
  => Assert.True(svc.Refund(o, 150m).IsError);

[Fact] void Refund_Unpaid_Order_Fails()
  => Assert.True(svc.Refund(unpaid, 10m).IsError);

// illustrative example
```

Bridge to Module 04: Spec-Driven Development turns this idea into a workflow — specs first, with Spec Kit or OpenSpec. Today you use specs and tests as context; next time you make them the source of truth.

Agentic Context: Let the Agent Fetch What It Needs

Beyond pre-loading: agents can retrieve just in time, so the window holds only what is needed right now.

Pre-loaded (up front)

You decide in advance

- Attach files and docs
- Always-on instructions
- Good: predictable, fast
- Risk: bloat, stale, wasted

Just-in-time (on demand)

Agent searches, reads, calls tools

- Search → open file → read range
- MCP tools fetch live data
- Good: small window, fresh
- Risk: more steps, needs good tools

THREE AGENTIC CONTEXT PATTERNS

Skills — progressive disclosure

Only name + description load first; full instructions load when the task matches.

Memory & notes files

The agent writes plan.md or notes.md outside the window and reads them back later.

MCP tools

Live data (issues, docs, DB) comes in as a small tool result, not a pasted dump.

Best practice: hybrid wins: pre-load the small stable core (instructions, spec); let the agent retrieve the rest on demand.

Context Engineering Best Practices Checklist

Run through this before you blame the model. Most bad answers trace back to one unchecked box.

SELECT

- [ ] Can I say why each attached file is there?
- [ ] Is the source of truth attached (spec, interface, tests)?
- [ ] Did I select the range, not the whole file?

LAYER

- [ ] Are stable rules in instructions, not chat?
- [ ] Are always-on files short and current?
- [ ] Is task detail in the prompt, not in global files?

COMPRESS

- [ ] Did I summarise after each decision?
- [ ] Are logs trimmed to the failing lines?
- [ ] Is the goal + constraints still in the window?

SCOPE

- [ ] New chat for each new task?
- [ ] Is noisy exploration done in a subagent?
- [ ] Is the critical rule first or last?

Keep this card: print it, pin it in your team channel, or turn it into a prompt file that reviews your context before you send a big agent task.

Lab: From Prompt-Only to Engineered Context

LAB 1 · 25 min

One task, five rounds. Change one thing per round and measure what happens.

Goal Implement the same small feature five ways and see how context choices change quality, tokens, iterations and time.

STEPS

1. R0 — baseline: new chat, prompt only (“add refund support to orders”). Record tokens, iterations, time, quality.
2. R1 — select: new chat; select the method and attach the 2–3 relevant files with #file. Same prompt.
3. R2 — layer: add a short instruction file and a 10-line spec plus failing tests. Re-run.
4. R3/R4 — compress and scope: summarise into a handoff note, start a fresh chat with only note + spec. Compare all rounds.

FILES YOU CREATE

```
repo/
lab03/
  .github/copilot-instructions.md
  specs/refund-spec.md
  notes/handoff.md
  results.md
```

DONE WHEN

results.md holds a filled worksheet for all five rounds, and you can name which single change helped most — and which one did nothing.

Lab Worksheet: Measure Every Round

Fill one row per round. The pattern across rows is the lesson, not any single number.

Round	What changed	Input tokens	Iterations	Time (min)	Quality 1–5	Notes
R0	Prompt only					
R1	Add: selected code and #file attachments					
R2	Add: instructions, spec and tests					
R3	Add: compressed handoff note					
R4	Add: fresh scoped chat (note + spec only)					

Quality rubric (one point each, plus 1 base)

- Compiles and tests pass
- Follows team conventions
- No invented APIs or files
- Small, reviewable diff

Questions to answer after R4

- Which round gave the biggest quality jump?
- Did tokens rise, then fall? Why?
- Which added context did nothing?

What You Can Do Now

Six outcomes from today — and the one habit that ties them together.

- ✓ Prompts are structured: task, role, constraints, example, output format
- ✓ The decompose → iterate → verify loop is used instead of one-shot prompts
- ✓ Context sources are selected deliberately with #file, #codebase and selections
- ✓ Stable rules live in instruction files; volatile detail stays in the prompt
- ✓ History is compressed and tasks are scoped with new chats or subagents
- ✓ Context effects are measured on tokens, iterations, time and quality

NEXT: MODULE 04 — SPEC-DRIVEN DEVELOPMENT WITH SPEC KIT / OPENSPEC · 2 HOURS

Move from using specs as context to making them the source of truth: write the spec first, then plan, tasks and implementation.

THE CONTEXT ENGINEERING LOOP

Prompt → Select → Layer → Compress → Scope → Measure

Questions & Discussion

Module 03 — Prompt Engineering Recap & Context Engineering